

Desarrollo Backend

Git, Repositorios, Branches y Trabajo Colaborativo

Material teórico para estudiantes

1. Objetivos de aprendizaje

- Comprender qué es un sistema de control de versiones y qué problemas resuelve.
- Diferenciar Git de plataformas como GitHub.
- Comprender repositorio local, repositorio remoto, working directory y staging area.
- Crear commits claros y consultar el historial.
- Crear, cambiar y eliminar branches.
- Sincronizar cambios mediante push, pull y fetch.
- Comprender el propósito de los Pull Requests y el code review.
- Entender qué es un merge conflict y cómo resolverlo.
- Aplicar un flujo de trabajo básico de desarrollo backend.

2. ¿Por qué necesitamos control de versiones?

En un proyecto backend varias personas pueden modificar simultáneamente controladores, servicios, modelos, tests y archivos de configuración. Sin un sistema de control de versiones, resulta difícil saber qué cambió, quién hizo el cambio, cuándo se hizo y cómo volver a una versión anterior.

Git permite mantener un historial de cambios y trabajar en paralelo mediante branches. Esto reduce el riesgo de que el trabajo de un desarrollador sobrescriba accidentalmente el de otro.

3. ¿Qué es Git?

Git es un sistema distribuido de control de versiones. Cada desarrollador puede tener una copia completa del repositorio y de su historial en su computadora.

Git permite, entre otras cosas:

- Registrar cambios.
- Consultar el historial.
- Crear líneas de desarrollo independientes.
- Comparar versiones.
- Integrar cambios.
- Recuperar estados anteriores del proyecto.

4. Git y GitHub no son lo mismo

Git es la herramienta de control de versiones. GitHub es una plataforma que permite alojar repositorios Git y agregar capacidades de colaboración, como Pull Requests, revisión de código, issues y automatizaciones.

También existen otras plataformas, como GitLab y Bitbucket.

5. ¿Qué es un repositorio?

Un repositorio es un proyecto administrado por Git. Contiene los archivos del proyecto y la información necesaria para conservar su historial.

Un proyecto backend podría tener una estructura como:

```
BackendApi/  
├── Controllers/  
├── Services/  
├── Models/  
├── Data/  
├── Tests/  
├── Program.cs  
├── appsettings.json  
├── README.md  
└── .gitignore
```

6. Repositorio local y remoto

El repositorio local está en la computadora del desarrollador. El repositorio remoto normalmente está alojado en una plataforma como GitHub. Ambos pueden intercambiar información mediante comandos como push, pull y fetch.

Una forma simple de visualizarlo:

Computadora → Git → Repositorio local → push/pull/fetch → Repositorio remoto

7. Working Directory, Staging Area y Repository

Git utiliza varias etapas conceptuales. El Working Directory contiene los archivos que estamos modificando. La Staging Area contiene los cambios que seleccionamos para el próximo commit. El repositorio local contiene los commits que ya forman parte del historial.

Flujo fundamental:

Working Directory → git add → Staging Area → git commit → Repository

8. git status

Permite conocer el estado actual del repositorio.

Ejemplo:

```
git status
```

9. git add

Agrega cambios a la Staging Area. No crea un commit.

Ejemplos:

```
git add Program.cs
```

```
git add .
```

10. Commit

Un commit representa un punto en la historia del proyecto. Contiene los cambios preparados y un mensaje que permite identificar su propósito.

Ejemplo:

```
git commit -m "Add users endpoint"
```

Los mensajes deberían ser concretos. Un mensaje como 'changes' aporta poca información; 'Add users endpoint' permite entender qué se modificó.

11. Historial de commits

Para consultar el historial:

```
git log
```

```
git log --oneline
```

12. ¿Qué es una branch?

Una branch es una línea de desarrollo. Permite realizar cambios de forma aislada respecto de otra línea, por ejemplo main.

En un proyecto real podemos tener main para la línea principal y branches como feature/login, feature/users o bugfix/payment.

13. ¿Por qué trabajar con branches?

- Aislar una nueva funcionalidad.
- Trabajar en paralelo con otros desarrolladores.
- Evitar modificar directamente la línea principal.
- Facilitar la revisión de código.
- Reducir el riesgo de introducir cambios incompletos en main.

14. Crear y cambiar branches

Ver branches:

git branch

Crear y cambiar:

git switch -c feature/login

Cambiar a una branch existente:

git switch main

15. El flujo de una funcionalidad

Un flujo sencillo para una funcionalidad backend es:

1. Actualizar main.
2. Crear una branch para la tarea.
3. Implementar y probar.
4. Crear uno o más commits.
5. Subir la branch al remoto.
6. Crear un Pull Request.
7. Realizar code review.
8. Integrar los cambios.

Ejemplo de comandos:

```
git switch main
git pull
git switch -c feature/users
# desarrollar y probar
git add .
git commit -m "Add users endpoint"
git push -u origin feature/users
```

16. ¿Qué significa origin?

origin es el nombre convencional que Git asigna al repositorio remoto principal cuando clonamos un repositorio. Podemos comprobarlo con:

```
git remote -v
```

17. git push

Envía commits de una branch local al repositorio remoto.

Ejemplo:

```
git push -u origin feature/users
```

Después de establecer la relación entre la branch local y la remota, normalmente podemos usar simplemente git push.

18. git pull

Obtiene cambios del repositorio remoto y actualiza la branch local actual. Es habitual utilizarlo antes de comenzar a trabajar sobre una branch que debe estar actualizada.

Ejemplo:

```
git switch main  
git pull
```

19. git fetch

Descarga información nueva del repositorio remoto sin integrar automáticamente esos cambios en la branch local actual.

Una diferencia conceptual:

fetch = 'quiero conocer/descargar lo que existe en el remoto'.

pull = 'quiero traer esos cambios y actualizar mi branch'.

20. Pull Request

Un Pull Request (PR) es una solicitud para incorporar los cambios de una branch a otra, normalmente mediante una plataforma como GitHub. No es un comando propio de Git.

Un PR permite revisar código, comentar, ejecutar validaciones y decidir si los cambios están listos para integrarse.

21. Code Review

El code review consiste en revisar los cambios realizados por otro desarrollador antes de integrarlos. El objetivo no es solamente encontrar errores: también ayuda a mejorar diseño, legibilidad, mantenibilidad, tests y consistencia con las convenciones del proyecto.

22. Merge

Merge es la operación mediante la cual Git combina los cambios de una línea de desarrollo con otra.

Conceptualmente:

feature/users → main

23. Merge Conflict

Un conflicto ocurre cuando Git no puede determinar automáticamente cómo combinar dos cambios. Un caso típico es que dos desarrolladores modifiquen las mismas líneas de un archivo de forma incompatible.

Git puede marcar el archivo con separadores como:

Desarrollo Backend — Git y trabajo colaborativo

```
<<<<<<< HEAD
return Ok(users);
=====
return Ok(activeUsers);
>>>>>>> feature/users
```

El desarrollador debe decidir cuál es la versión correcta o construir una tercera solución.

Después de resolverlo:

```
git add .
git commit -m "Resolve merge conflict"
```

24. .gitignore

El archivo .gitignore indica qué archivos o carpetas Git no debería incluir en el control de versiones.

En proyectos .NET suele ser importante excluir archivos generados por compilación, carpetas temporales y, según el caso, archivos locales que contienen configuración sensible.

Importante: .gitignore no elimina un archivo que ya fue agregado y commiteado al repositorio.

25. Buenas prácticas

- No trabajar directamente sobre main salvo que el flujo del proyecto lo requiera explícitamente.
- Crear branches con nombres descriptivos, por ejemplo feature/products o bugfix/login.
- Realizar commits pequeños y coherentes.
- Escribir mensajes de commit claros.
- Actualizar la branch antes de comenzar una tarea.
- No subir contraseñas, tokens, claves privadas ni secretos al repositorio.
- Revisar los cambios antes de hacer commit con git status y git diff.
- Mantener el código compilando y los tests funcionando cuando corresponda.
- Usar Pull Requests para facilitar la revisión y trazabilidad.

26. Git Cheat Sheet

Clonar: git clone <url>

Estado: git status

Agregar cambios: git add .

Commit: git commit -m "message"

Historial: git log --oneline

Ver branches: git branch

Crear branch: git switch -c feature/name

Cambiar branch: git switch main

Traer cambios: git pull

Descargar información remota: git fetch

Subir cambios: git push

Ver remoto: git remote -v

Ver diferencias: git diff

27. Ejemplo completo para una API .NET

Supongamos que necesitamos agregar GET /products. Un flujo razonable sería:

```
git switch main
git pull
git switch -c feature/products
# implementar GET /products
# ejecutar la API y probar el endpoint
git status
git add .
git commit -m "Add products endpoint"
git push -u origin feature/products
```

Luego se crea un Pull Request desde feature/products hacia main, se revisa el código y, si todo está correcto, se integran los cambios.

28. Autoevaluación

1. ¿Cuál es la diferencia entre Git y GitHub?
2. ¿Qué diferencia existe entre Working Directory y Staging Area?
3. ¿Qué hace git add?
4. ¿Qué representa un commit?
5. ¿Por qué usamos branches?
6. ¿Qué diferencia existe entre git pull y git fetch?
7. ¿Qué es un Pull Request?
8. ¿Qué es un merge conflict?
9. ¿Por qué no debemos subir secretos al repositorio?
10. Describí el flujo completo desde la creación de una branch hasta su integración mediante Pull Request.

29. Conceptos clave para recordar

Git permite controlar la evolución del código. Un commit representa un punto del historial. Una branch permite desarrollar de manera aislada. Un repositorio remoto permite compartir el trabajo. Un Pull Request facilita la revisión y la integración. Los conflictos no significan que Git haya fallado: significan que necesita que un desarrollador tome una decisión sobre cambios incompatibles.

Flujo esencial: clone → branch → code → add → commit → push → Pull Request → review → merge.